Lab03

Max Stróżyk

Winter 2024/2025

English translation of the Polish original. Historical results and formulas retained.

Contents

1 Task 1 **2**
1.1 Overview of the Householder method . . . 2
1.2 Results and conclusions . . . 2

2 Task 2 **3**
2.1 Overview of QR decomposition . . . 3
2.2 Results and conclusions: . . . 3

3 Task 3 **4**
3.1 Matrix 2-norm: overview and comparison . . . 4
3.2 Results and conclusions: . . . 4

4 Task 4 **5**
4.1 Iterative methods: overview and comparison . . . 5
4.2 Results and conclusions: . . . 5
4.2.1 Parameter k . . . 5
4.2.2 Parameter c . . . 6
4.3 Suggestions . . . 6

1 Task 1

1.1 Overview of the Householder method

The Householder method is used to orthogonalize vectors and reduce matrices to simpler forms, such as upper-triangular or Hessenberg matrices. It is widely used in numerical algorithms, including QR decomposition and eigenvalue computation.

The main idea is to apply a Householder transformation: a reflection matrix transforms a given vector into a simpler form.

$$P = I - 2\frac{vv^T}{v^Tv}, \quad Q = I - 2\frac{uu^T}{u^Tu}$$

1.2 Results and conclusions

The table gives Householder-method test results for different matrix sizes n and relationships between vectors e and x. Errors are computed as follows:

- **Error 1:** Checks the error of the unitary matrix P, that is, $P^TP = I$.
- **Error 2:** Checks the error of the unitary matrix Q, that is, $Q^TQ = I$.
- **Error 3:** Compares Px with $-ke$, where k defines the scaling.
- **Error 4:** Checks whether Qx equals $-ke$.

The table values represent maximum deviations from ideal results and indicate the algorithm's precision:

n	k	Error 1	Error 2	Error 3	Error 4
3	1.0891	$2.2203e-16$	$2.2205e-16$	$2.2204e-16$	$2.2204e-16$
3000	0.9830	$1.7765e-15$	$1.9983e-15$	$3.2197e-15$	$2.9975e-15$
3000; $e \sim x$	1.000	$1.9983e-15$	$1.8875e-15$	$2.6646e-15$	$3.8857e-15$
3000; e, x - orthogonal	0.9938	$1.9985e-15$	$1.7763e-15$	$4.2187e-15$	$3.5528e-15$

Conclusions:

- **Precision for small n:** For small values of n, such as $n = 3$, the Householder method is highly accurate, with errors of order 10^{-16}. This indicates nearly ideal unitarity of P and Q and an accurate mapping of the resulting vector.
- **Stability for large n:** For large matrices ($n = 3000$), the method remains numerically stable, although errors increase slightly.
- **Effect of the relationship between e and x:** When e is close to x, the errors are negligible.
- **Orthogonality of e and x:** Errors are negligible for orthogonal vectors.

2 Task 2

2.1 Overview of QR decomposition

QR decomposition factors a matrix A into the product of two matrices:

$$A = QR,$$

where Q is orthogonal ($Q^TQ = I$) and R is upper triangular. The method is widely used in numerical computation, particularly for computing matrix eigenvalues. Whereas the power method repeatedly multiplies A by a vector and normalizes the result, the QR algorithm operates on a complete vector basis. QR decomposition leads to basis orthogonalization and vector normalization across successive iterations.

2.2 Results and conclusions:

Numerical-error results for QR decomposition are as follows:

Error definitions:

- **Error 1:** Checks the unitarity of Q, that is, $Q^TQ = I$. For an ideal QR decomposition, Q should be orthogonal.
- **Error 2:** Checks whether the product of Q and R reconstructs the original matrix A, that is, $QR = A$. This verifies the decomposition itself.
- **Error 3:** Checks the unitarity of $Q1$, obtained by further transformations of Q. For orthogonal matrices, $Q1$ should also be orthogonal.
- **Error 4:** Checks whether the product of $Q1$ and $R1$ reconstructs A. As for error 2, this verifies the decomposition after additional transformations.

The table gives numerical errors for different matrix sizes:

n	error-1	error-2	error-3	error-4
3	$8.8817e-16$	$5.5512e-17$	$4.4408e-16$	$3.3308e-16$
500	$5.4369e-13$	$3.5417e-14$	$1.1103e-15$	$7.9382e-15$
100	$3.9967e-15$	$5.6844e-14$	$1.7765e-15$	$3.1265e-13$

Conclusions:

- For small matrices ($n = 3$), numerical errors are very small, indicating high precision of the QR method at small matrix sizes.
- As matrix size increases ($n = 500$), errors grow. This may reflect accumulated rounding errors and limited numerical precision, which become more visible for large matrices. The errors remain small but are significantly larger.
- For the symmetric matrices considered in the last table row, errors decrease, suggesting better numerical stability of the QR method for these matrices.
- Overall, the QR method has good numerical stability, particularly for symmetric matrices, where convergence is clearer and errors are smaller.

3 Task 3

3.1 Matrix 2-norm: overview and comparison

The matrix 2-norm (spectral norm) is defined as the largest singular value of A. It equals the square root of the largest eigenvalue of $A^T A$, allowing the distribution of matrix values to be assessed.

Our algorithm computes this norm by finding roots of the matrix's characteristic polynomial. This approach is sensitive to numerical errors, particularly when eigenvalues are close together. Polynomial-root operations can produce unwanted effects, including complex error values caused by rounding or numerical instability.

The *power method* can normally be used to compute the matrix 2-norm. This iterative technique provides a stable approximation of the largest eigenvalue. Its steps are:

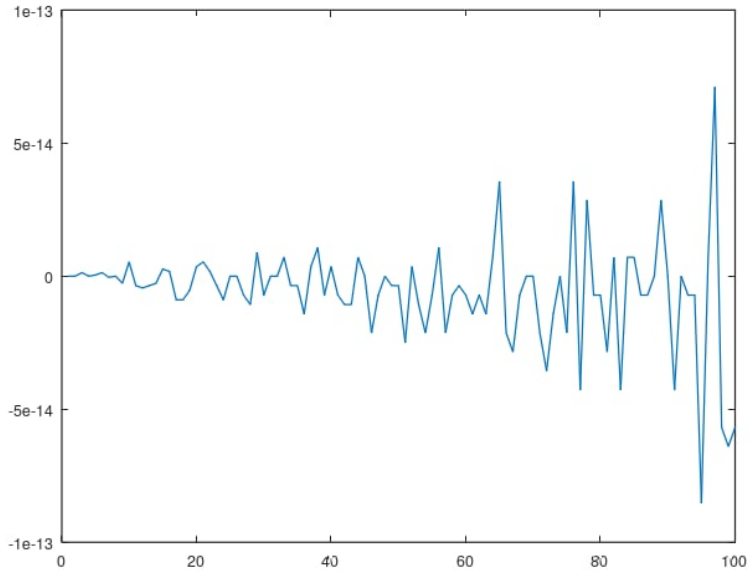
1. Start with a vector v_0 of arbitrary values.
2. At each iteration, compute $v_{k+1} = Av_k$ and normalize the resulting vector.
3. Repeat until the iterations converge to the eigenvalue with the largest absolute value.

This method is more stable than operations on the characteristic polynomial because it avoids rounding-error problems and does not require finding polynomial roots.

3.2 Results and conclusions:

Comparing our norm2 function with Octave's built-in norm, we observe discrepancies due to numerical errors. Our implementation is more sensitive to errors in characteristic-polynomial roots, particularly for matrices with similar eigenvalues. Large diagonal values can also cause rounding errors because arithmetic precision is limited.

The following plot illustrates errors in our implementation.



Conclusions:

- Error values fluctuate substantially. The algorithm is numerically unstable.

4 Task 4

4.1 Iterative methods: overview and comparison

This section analyzes three iterative methods for solving linear systems: Jacobi, Gauss–Seidel and SOR (Successive Over-Relaxation). Each transforms the system matrix A into a form suitable for iterative solution approximation.

- **Jacobi method**: Split A into $A = D + R$, where D is diagonal and R contains the off-diagonal entries. The iteration is:
$$x^{(k+1)} = D^{-1}(b - Rx^{(k)}).$$
This method uses only the values of $x^{(k)}$ from the preceding iteration.

- **Gauss–Seidel method**: Decompose $A = L + U$, where L is lower triangular including the diagonal, and U is upper triangular. The iteration is:
$$x^{(k+1)} = L^{-1}(b - Ux^{(k+1)}).$$
New values of $x^{(k+1)}$ are used immediately in the current calculation, accelerating convergence.

- **SOR method**: This generalizes Gauss–Seidel by introducing a relaxation factor ω. The formula is:
$$x^{(k+1)} = (1 - \omega)x^{(k)} + \omega L^{-1}(b - Ux^{(k+1)}).$$
Choosing ω appropriately is essential for efficiency.

4.2 Results and conclusions:

For all calculations we set $x = 0, b = rand(n, 1) * 10$, perform 1000 trials and average their results.

4.2.1 Parameter k

We estimate k, the number of iterations required to establish convergence. To avoid errors from a lack of diagonal dominance, we set $c = n \cdot 10$. This gives a dominant diagonal and removes errors due to matrix structure. We can then set $n = 10$, since this diagonal magnitude avoids size-related and convergence problems.

Method	1	3	5	10	16
Jacobi	$4.0556e-03$	$6.5079e-06$	$9.9949e-09$	$2.7341e-15$	$1.1819e-17$
Gauss-Siedel	$3.3702e-03$	$4.4534e-07$	$4.7716e-12$	$2.6600e-17$	$1.8228e-17$
SOR	0.010987	$1.3029e-04$	$1.2422e-06$	$2.5774e-11$	$2.3522e-17$

Conclusions:

- SOR: effect of a poorly chosen relaxation factor:
Although SOR should theoretically converge fastest, our calculations do not confirm this. A fixed, poorly chosen relaxation factor was used, reducing its efficiency. A poor relaxation factor can therefore substantially reduce the method's effectiveness.

- Comparison of Jacobi and Gauss–Seidel:
The results show that Gauss–Seidel converges faster from the first iteration. New values computed within an iteration are immediately used in subsequent calculations. Jacobi uses only preceding-iteration values, slowing convergence.

4.2.2 Parameter c

We estimate c, the amount added to the diagonal in $A + c \cdot \text{eye}(n)$. We set $k = 100$ to ensure convergence.

- Case $c = n/2$

Method	3	10	100
Jacobi	$1.4872e-08$	$2.0980e-03$	$8.8618e-03$
Gauss-Siedel	$5.8098e-16$	$3.9518e-16$	$17138e-16$
SOR	$5.9233e-16$	$3.9828e-16$	$17124e-16$

- Case $c = 10 * n$

Method	3	10	100
Jacobi	$3.9344e-17$	$2.5617e-17$	$1.0956e-17$
Gauss-Siedel	$4.0613e-17$	$2.5840e-17$	$1.0976e-17$
SOR	$3.7866e-17$	$2.5573e-17$	$1.0902e-17$

Conclusions:

- The approximation of c suggested by the mean $E[\sum_i^n X_i]$, where $X_1 \sim \text{U}([0,1])$, appears to be a good estimate. The Jacobi errors must arise from the algorithm itself, but I do not understand why they occur for [sentence unfinished in the original].

4.3 Suggestions

Stopping criterion: Choose p and define a stopping time at step n when the following condition holds:

$$\left| \frac{x_n - x_{n-1}}{x_n} \right| \times 100 < p$$